NeuroGrip

an EEG Signal controlled Robotic Arm

22MAT230 - Mathematics for Computing 4

22AIE214 - Introduction to AI Robotics

AIE - B Team 007

Anto Rishath	CB.SC.U4AIE23103
Arunoday A K	CB.SC.U4AIE23106
Achutharaj R	CB.SC.U4AIE23108
Vijay Santhosh P	CB.SC.U4AIE23157
Yashwanth P T	CB.SC.U4AIE23164

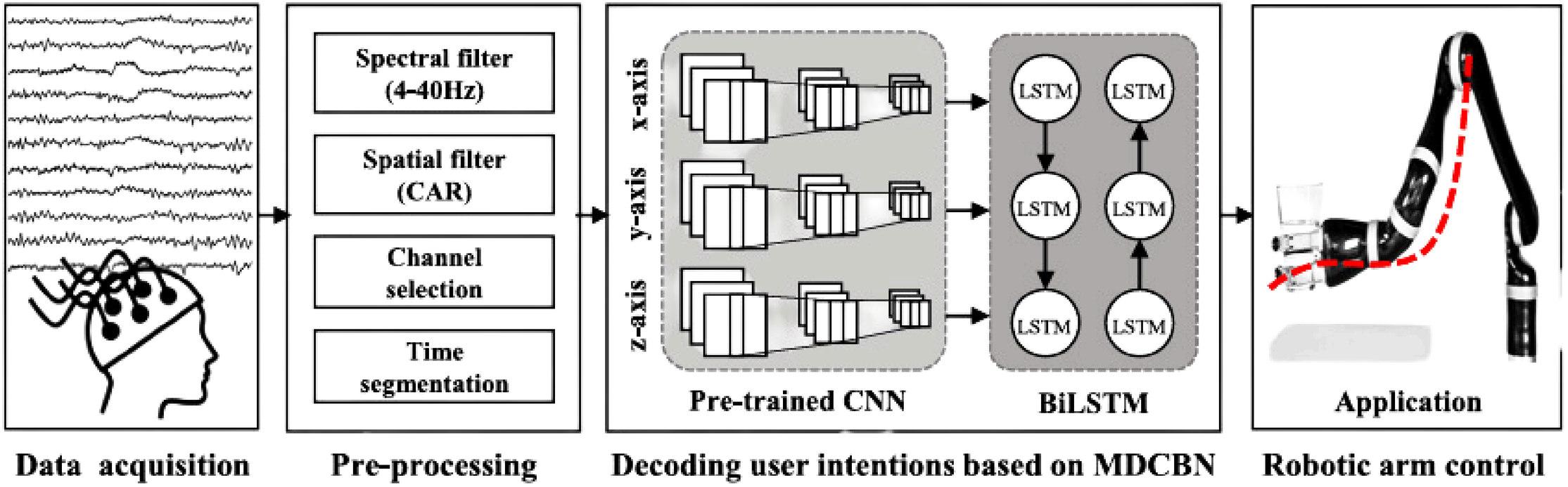
Problem Statement

Individuals with motor impairments face significant challenges in performing daily activities due to limited or lost motor control. While Brain-Computer Interface (BCI) systems have shown promise in translating neural signals into actionable commands, most existing solutions are confined to laboratory settings, lacking real-time processing and practical deployment in assistive robotics. There is a need for a non-invasive, real-time BCI system that can accurately decode motor intentions from EEG signals and translate them into precise control commands for robotic arms.

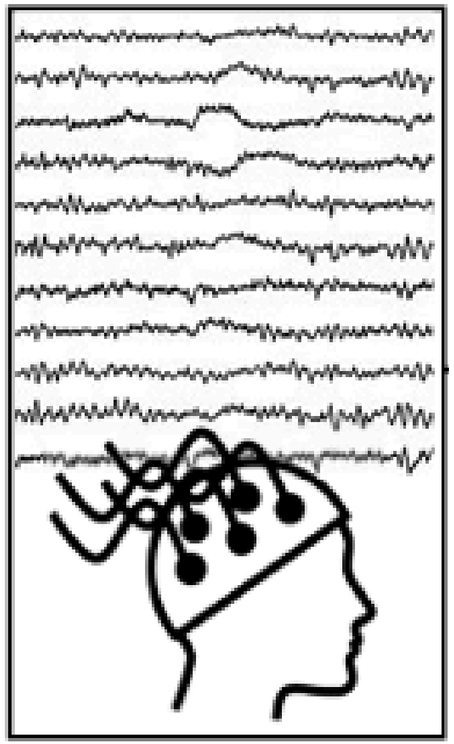
Introduction

NeuroGrip is an advanced EEG-based brain-machine interface (BMI) system designed to control a robotic arm using intuitive motor imagery. Inspired by the paper Brain-Controlled Robotic Arm System Based on Multi-Directional CNN-BiLSTM Network Using EEG Signals, this project leverages deep learning techniques to decode EEG signals and translate them into real-time robotic arm movements.

Workflow

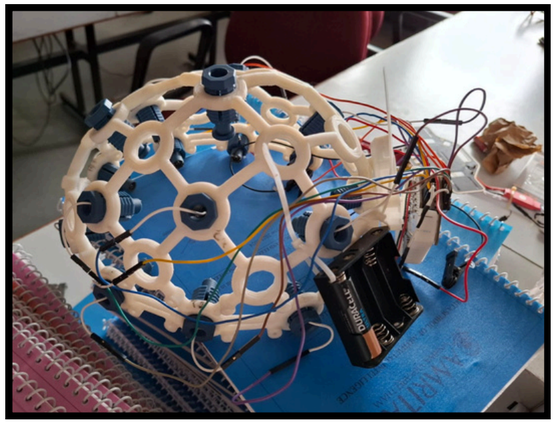

Spectral filter (4-40Hz)
Spatial filter (CAR)
Channel selection
Time segmentation
x-axis
y-axis
z-axis
LSTM
LSTM
LSTM
LSTM
LSTM
LSTM
Pre-trained CNN
BiLSTM
Application
Data acquisition
Pre-processing
Decoding user intentions based on MDCBN
Robotic arm control

1. Data Acquisition

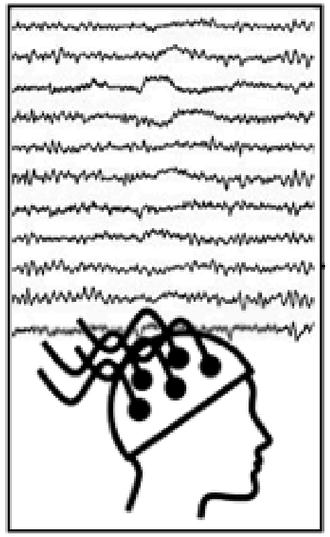


The data is obtained or collected from a machine called BMI (Brain Machine Interface)

BMI is a direct communication link between the brain's electrical signals and an external device.



1. Data Acquisition

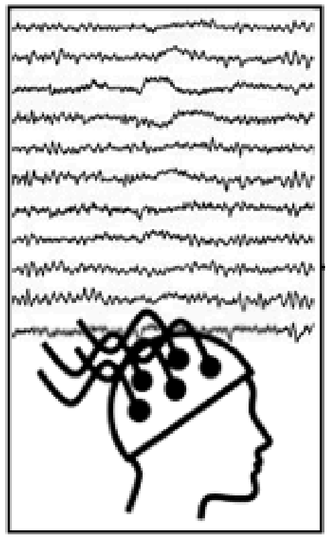


We are decoding the intuitive upper extremity motor imagery for multidirectional arm reaching tasks in 3D environment.

Moto imagery is the thought of moving specific body parts without any external stimuli.

When a user imagines the motion of a limb,
Event-related desynchronization / synchronization (**ERD** / **ERS**) features, which are referred to as a Sensory-motor rhythm (**SMR**), are induced in EEG signals over the primary sensorimotor area

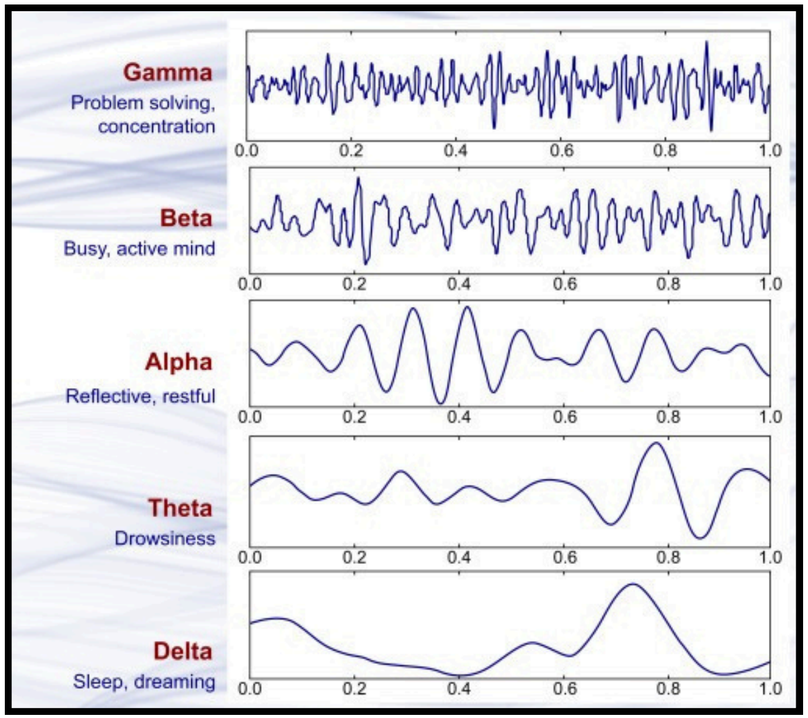
1. Data Acquisition



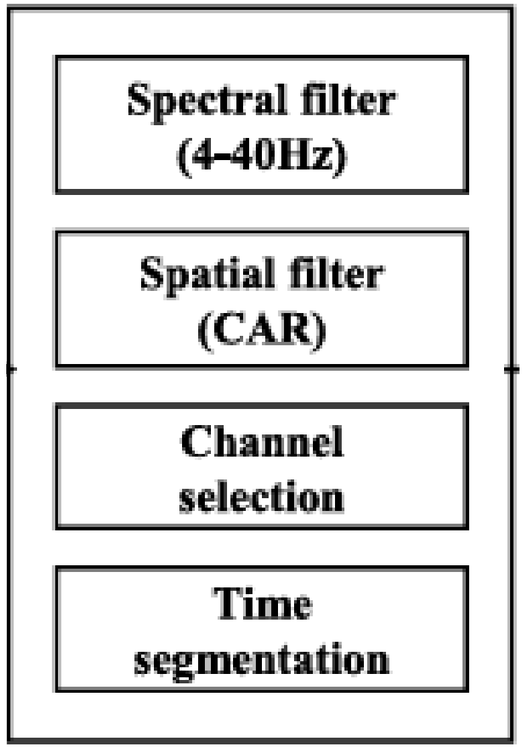
Brain waves are classified into 5 types :

We are going to use alpha(Mu) and beta waves, ranging in frequency of 8-12 Hz and 13- 30 Hz respectively

Mu waves are the oscillations in the alpha band that occurs over the primary motor cortex area



2. Pre-Processing

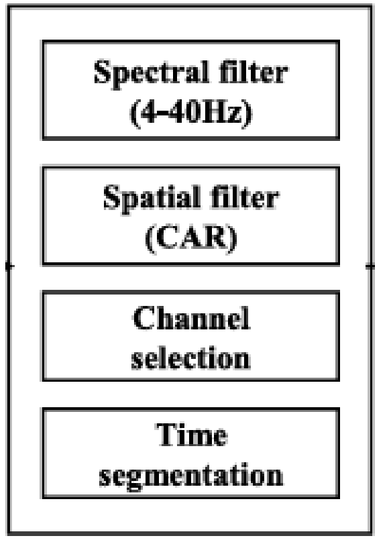


- EEG signals are recorded at 1,000 Hz, filtered (4-40 Hz), and downsampled.
- Kinematic data used to establish velocity profiles for training.

Artifact Removal:

- Independent Component Analysis (ICA) is used to remove artifacts (eye blinks, muscle noise, etc).
- Contaminated components similar to FP1 or FP2 channels are automatically removed.

2. Pre-Processing

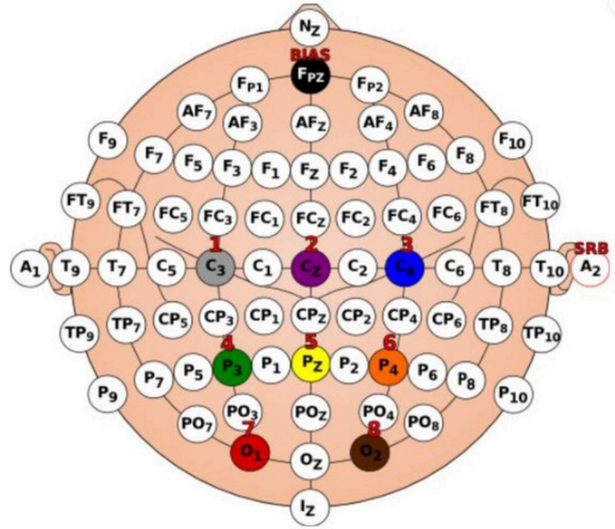


Re-referencing:

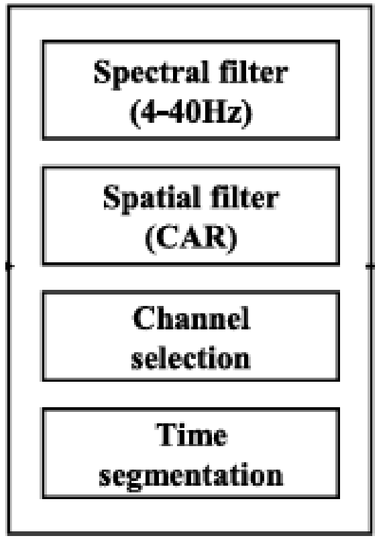
- We would be applying Common Average Reference (CAR) filter to improve spatial distribution.

Channel Selection:

- Focusing on 20 motor-related channels near the primary and supplementary motor cortices (FC5, FC3, FC1, FC2, FC4, FC6, C5, C3, C1, C2, C4, C6, CP5, CP3, CP1, CPz, CP2, CP4, and CP6).



2. Pre-Processing



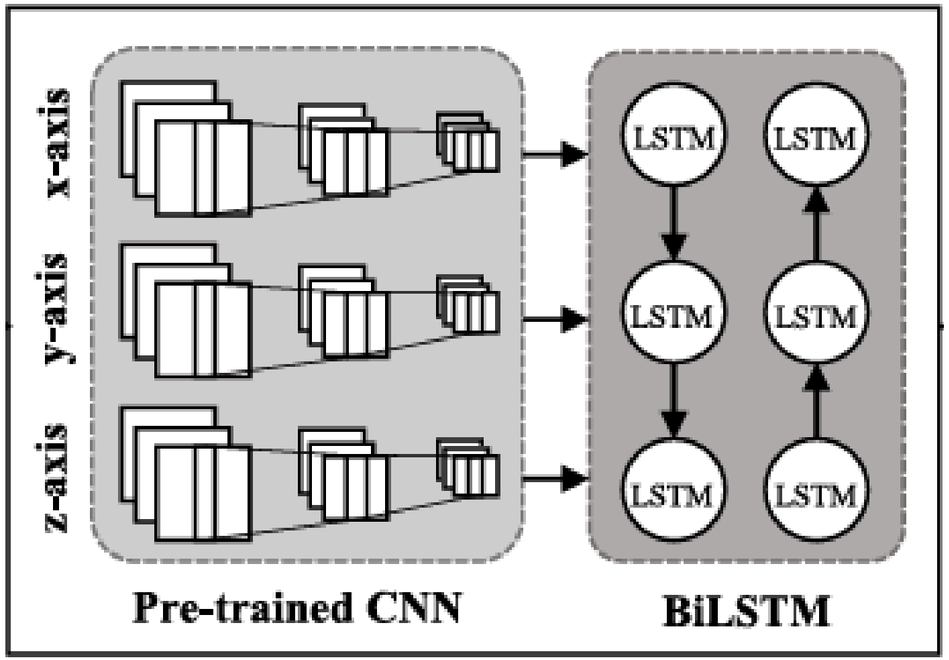
Downsampling:

- EEG signals reduced from 1,000 Hz to 100 Hz for computational efficiency.

Velocity Profile Extraction

- Baseline movement trajectories were recorded using the robotic arm.
- Movement velocity profiles were computed to serve as ground truth labels for deep learning model training.
- Each velocity profile was mapped to x, y, and z axes to enable 3D trajectory reconstruction.

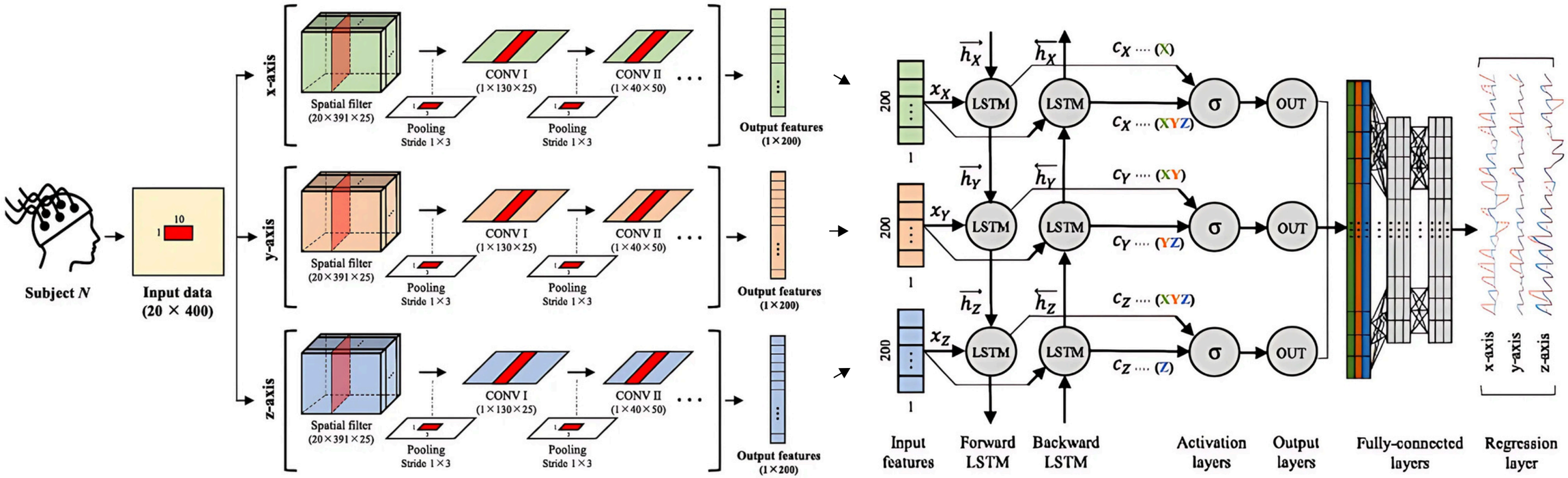
3. Model Training



MDCBN is a hybrid deep learning framework combining **CNN** and **BiLSTM** for decoding 3D arm movement from EEG signals.

It processes both spatial features (CNN) and temporal dependencies (BiLSTM) for improved movement prediction.

3. Model Training



3. Model Training

Architecture

CNN Pretraining:

Extracts multi-directional features per axis (x, y, z).
Uses DeepConvNet-based architecture for feature extraction.

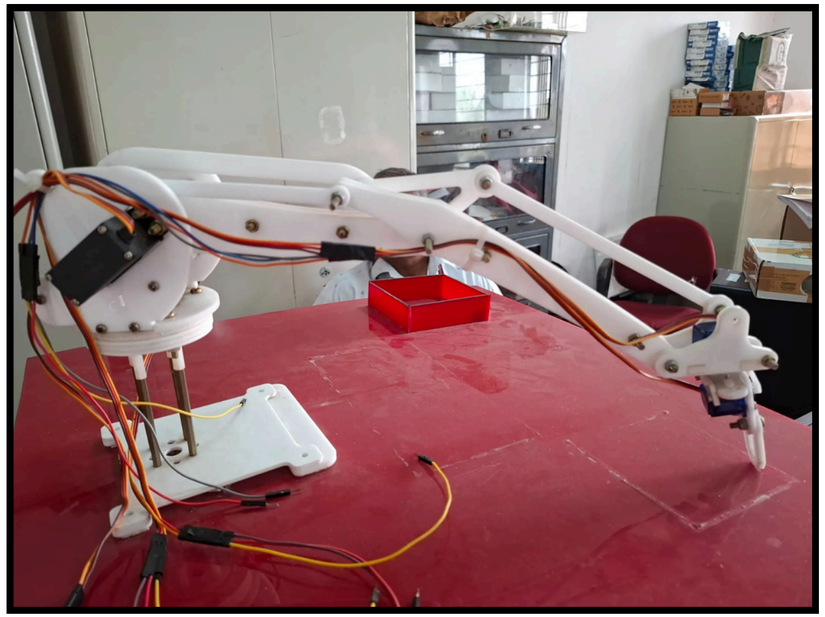
BiLSTM Network:

Captures relationships between axes for accurate 3D movement prediction.
Uses coarse-to-fine training (x → xy → xyz) for structured learning.
Forward & Backward LSTM layers ensure bidirectional feature learning.

Final Output:

Predicts velocity profiles for 3D robotic arm control.

4. Robotic Arm Control



Degrees of Freedom (DOF)

The robotic arm consists of multiple joints allowing movement across different axes:

- Base Rotation: 1 DOF (left/right rotation)
- Shoulder Joint: 1 DOF (up/down movement)
- Elbow Joint: 1 DOF (bending movement)
- Wrist Roll: 1 DOF (wrist rotation)
- Gripper (End Effector): 1 DOF (open/close function)

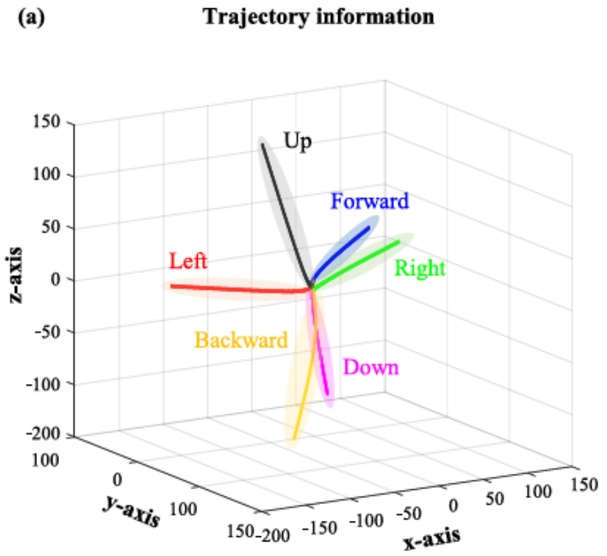
Total DOF: 5

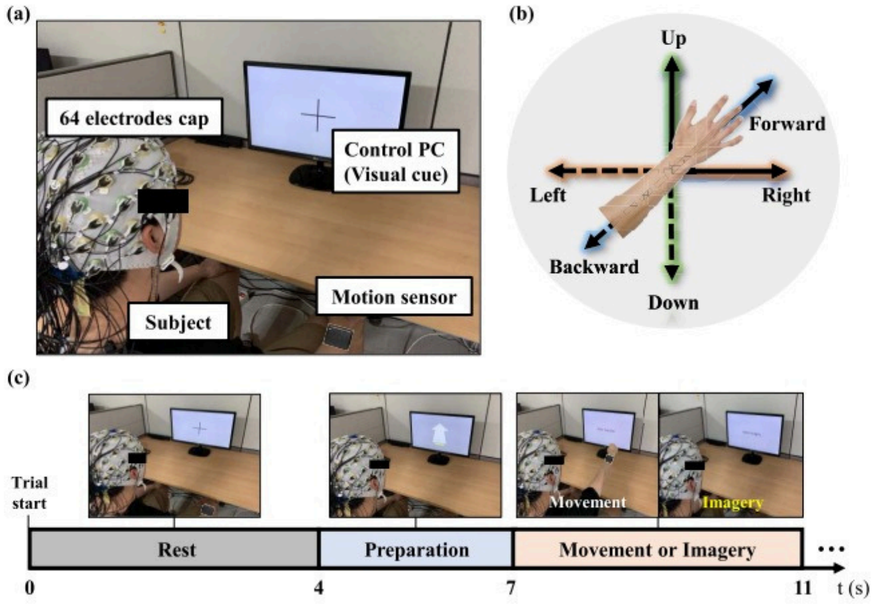
Experimental Setup

- Subjects sits in front of a monitor (~60 cm distance).
- Visual cues: '+' for rest, arrows for movement (left, right, forward, backward, up, down).
- Two sessions: Movement (actual) & Imagery (mental simulation).

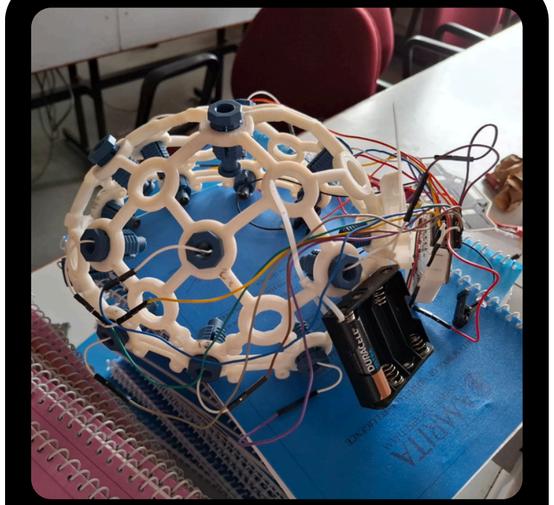
Procedure (240 trials per subject, 40 per direction):

- Resting (4s): Minimal movement & blinks.
- Cue Display (3s): Random directional instruction appears.
- Action Phase (4s): Perform actual movement or mental imagery.





Hardware integration



Data collection using OpenBCI

>



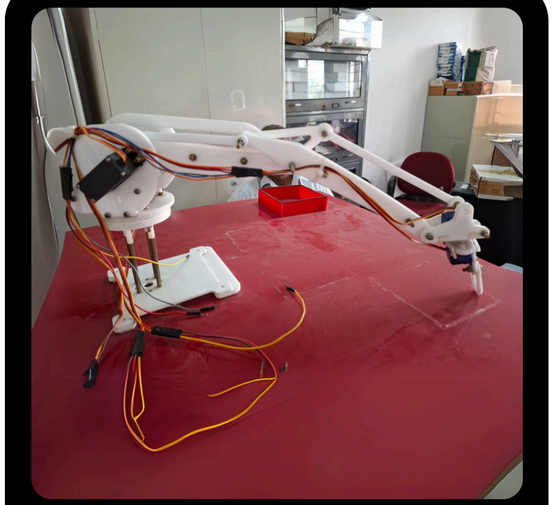
Data Pre-processing and Model Training

>



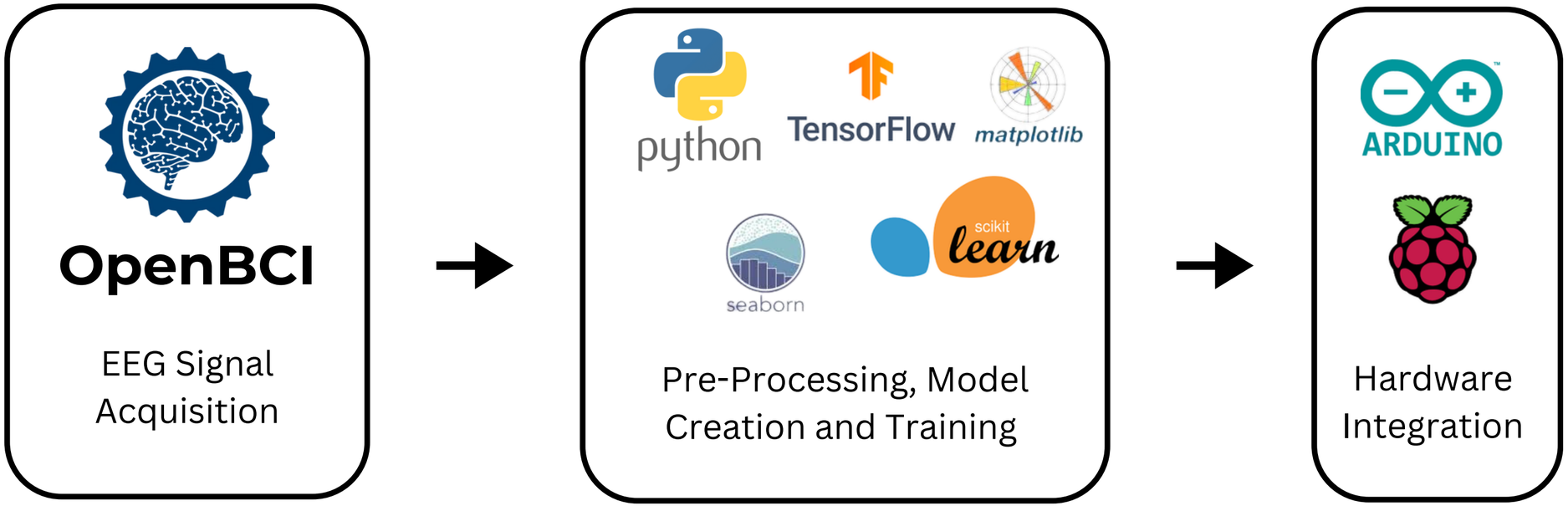
Instructions from Model Output to the Robotic Arm

>



Arm executes the received instructions

Tech Stack


OpenBCI
EEG Signal Acquisition
python
TensorFlow
matplotlib
seaborn
scikit learn
Pre-Processing, Model Creation and Training
ARDUINO
Hardware Integration
GAZEBO
Visualition

References

[1] **OpenBCI Documentation** https://openbci.com/community/use-your-imagination-power-to-control-robots-and-devices/?utm_source=chatgpt.com

[2] **OpenBCI Documentation** https://docs.openbci.com/Examples/EEGProjects/MotorImagery/?utm_source=chatgpt.com

[3] **Hardware Implementation** https://www.hackster.io/sbcomponentsuk/piarm-the-diy-robotic-arm-for-raspberry-pi-51e634?utm_source=chatgpt.com

[4] **Hardware Implementation** https://www.instructables.com/Automation-of-an-Articulated-Arm-With-Raspberry-Pi/?utm_source=chatgpt.com

[5] **Hardware Implementation** https://www.instructables.com/Building-robots-with-Raspberry-Pi-and-Python/?utm_source=chatgpt.com

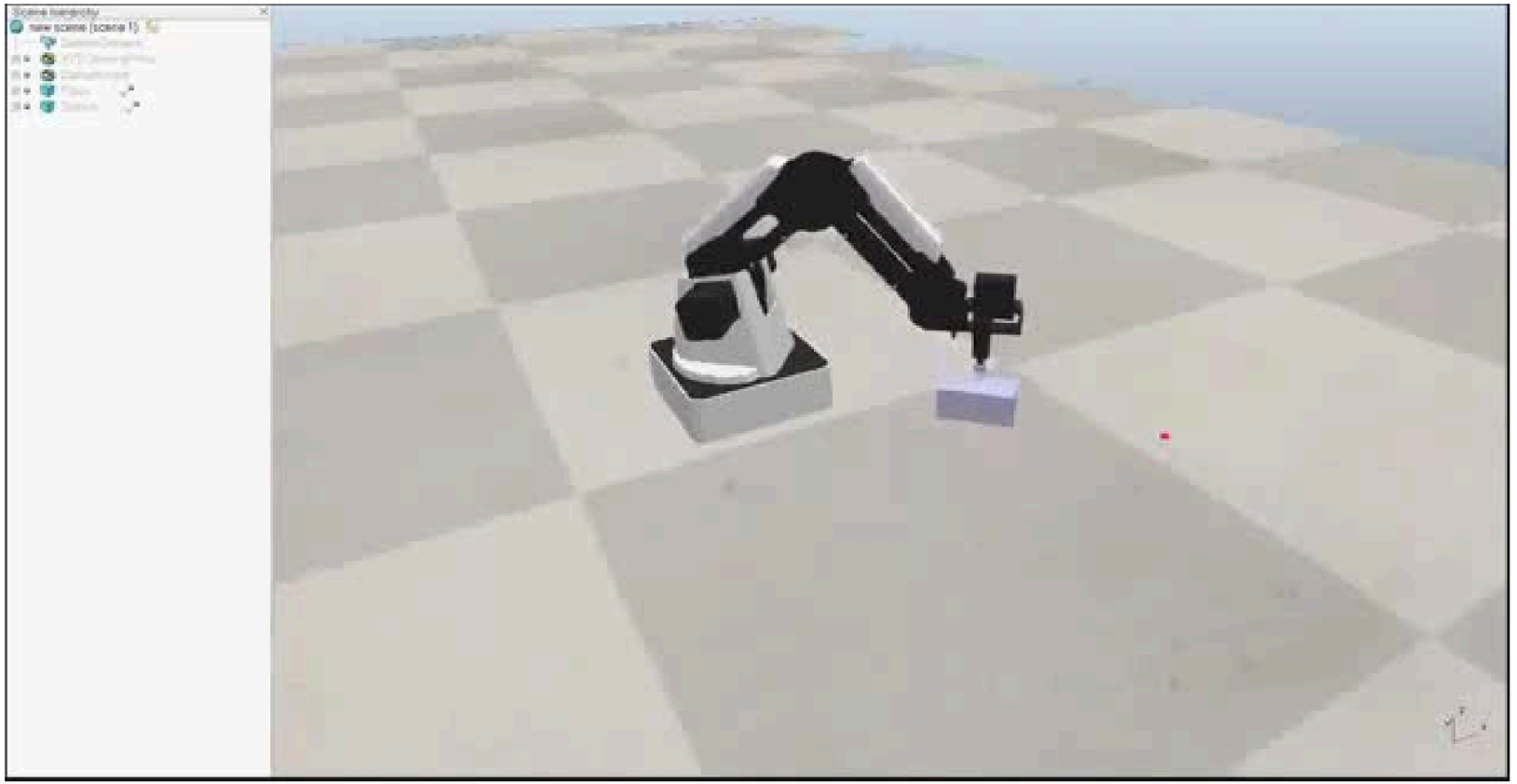
[6] **OpenBCI Documentation** https://openbci.com/forum/index.php?p=%2Fdiscussion%2F3080%2Fhow-to-run-the-openbci-gui-on-raspberry-pi&utm_source=chatgpt.com

[7] **Research Paper** https://ieeexplore.ieee.org/document/9040397?utm_source=chatgpt.com

Tasks Accomplished

- Method of data acquisition from OpenBCI helmet.
- Choosing signals for our application.
- Choosing specific pins for signal reading (20 pins).
- Choosing filters for data pre-processing (ICA).
- Choosing the model for classification task (MDCBN).
- How the hardware should be implemented to give instructions to the robotic arm.

Tasks Accomplished



Sample Visualization on Coppeliasim

Conclusion

This initial review confirms that our planned methodology is both viable and well-grounded in current research, as reflected in recent studies. We are now poised to move from the planning phase into the implementation phase, where we will address the practical challenges and refine our techniques through iterative testing and calibration

Thank You!